

Assignment 4: AI-Powered Chrome Extension for Requirement Analysis

Problem Statement

In software requirement meetings, stakeholders often provide incomplete, vague, or ambiguous information. Analysts usually miss important clarifications during live discussions, leading to poor-quality Functional Requirements (FR) and Non-Functional Requirements (NFR).

Develop an AI-powered Chrome Extension that listens to a live Zoom meeting transcript and generates intelligent clarification questions in real time. Based on the clarification questions and stakeholder responses, the system should generate refined Functional and Non-Functional Requirements.

The system should also evaluate and compare the quality of requirements generated:

- Without clarification support
 - With AI-generated clarification support through the Chrome Extension
-

Objective

The objective of this assignment is to:

- Understand AI-assisted Requirement Engineering in live meetings
 - Build a Chrome Extension for real-time transcript analysis
 - Detect ambiguous or incomplete stakeholder statements
 - Generate intelligent clarification questions automatically
 - Refine Functional and Non-Functional Requirements using stakeholder responses
 - Evaluate the impact of clarification on requirement quality
 - Compare requirement quality before and after clarification
-

Conversation for Analysis

Hiring Manager:

We need to build an AI-based resume analyzer that can automatically shortlist candidates for our software engineering roles.

ML Engineer:

Okay. How should the system decide which candidates to shortlist?

Hiring Manager:

It should rank them based on relevance to the job description.

ML Engineer:

How are we defining relevance?

Hiring Manager:

Mainly skills and experience. And overall profile strength.

ML Engineer:

What does overall profile strength include?

Hiring Manager:

Things like good companies, solid projects, impactful work.

ML Engineer:

Should we prioritize years of experience?

Hiring Manager:

Yes, but not strictly. Sometimes a strong fresher is better than someone with 5 average years.

ML Engineer:

Do we have historical hiring data to train the system?

Hiring Manager:

We have past resumes and hiring decisions, but they're not very structured.

ML Engineer:

How accurate should the system be?

Hiring Manager:

It should be good enough so that HR trusts it.

ML Engineer:

Do we need explainability? For example, why a candidate was ranked higher?

Hiring Manager:

Yes, that would be useful.

ML Engineer:

Are there any constraints regarding bias or fairness?

Hiring Manager:

Yes, we must avoid bias, especially related to gender or college background.

ML Engineer:

Should the system process resumes in real-time or batch mode?

Hiring Manager:

It shouldn't be slow.

ML Engineer:

What is the expected response time per resume?

Hiring Manager:

Ideally quick.

ML Engineer:

What is the timeline for delivery?

Hiring Manager:

We need an MVP soon.

Features to Implement

1. Develop a Chrome Extension for Zoom/meeting transcript monitoring
2. Capture or read live meeting transcripts
3. Detect vague, incomplete, or ambiguous requirements
4. Generate AI-based clarification questions in real time
5. Accept stakeholder responses to clarification questions
6. Extract Functional Requirements (FR)
7. Extract Non-Functional Requirements (NFR)
8. Categorize NFRs (Security, Performance, Reliability, Fairness, etc.)
9. Generate refined requirements after clarification
10. Compare requirements generated:
 - Without clarification
 - With clarification
11. Evaluate requirement quality using suitable metrics
12. Display generated FRs, NFRs, clarification questions, and evaluation results
13. Export generated requirements and evaluation reports as PDF/DOCX/TXT